



GoalFlow Technical Architecture

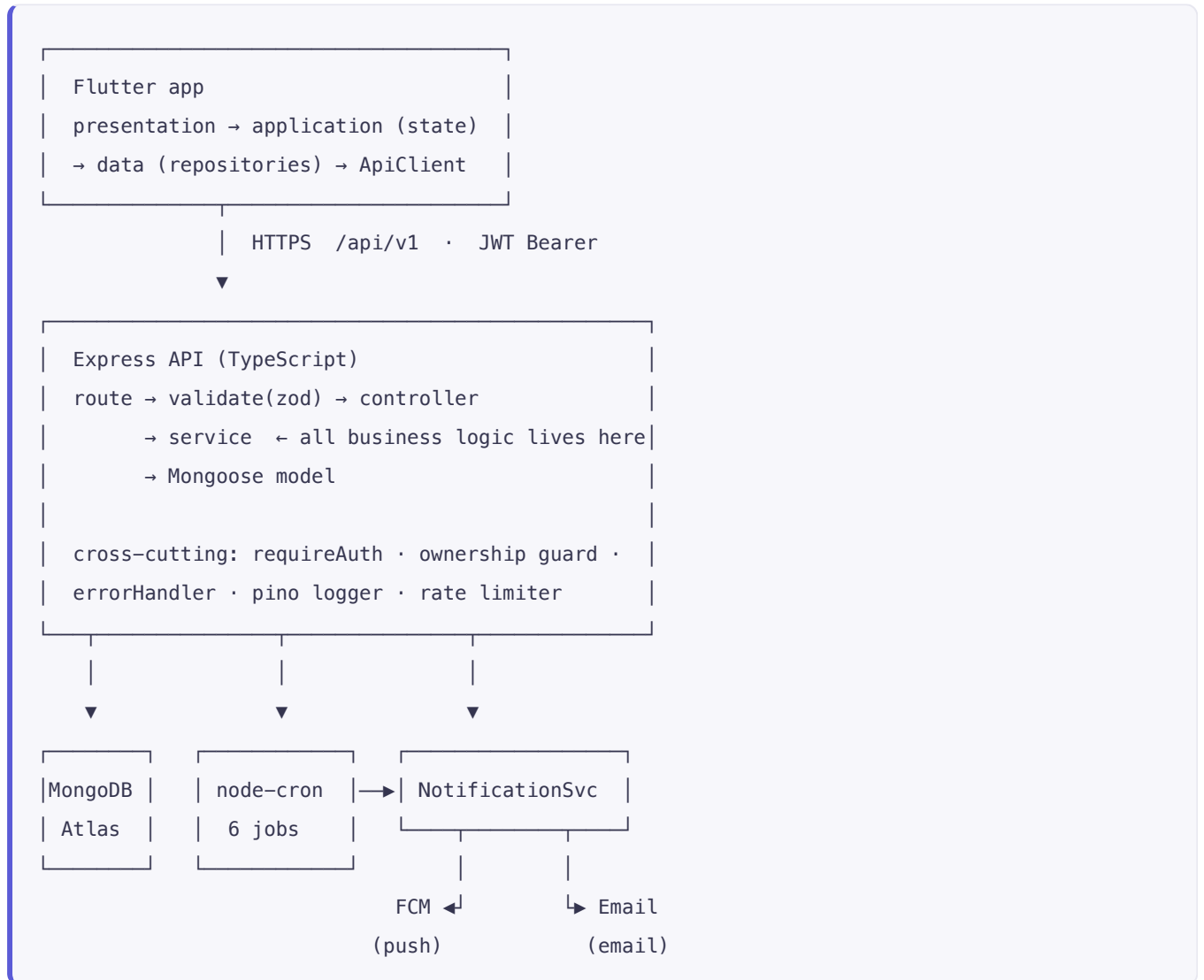
How the system is built and what every piece is used for

Stack Flutter · Node.js + TypeScript + Express · MongoDB Atlas · FCM · Email

Demo login demo@goalflow.app / Demo1234

How the system is put together, what each library is doing, and why it is there. Written so another developer can change something without first reading everything.

1. Shape of the system



One rule holds the whole thing together: business logic lives in services on the backend and in providers on the client. A route handler never computes anything, and a widget never calls the network.

2. Backend libraries — what each one actually does

Library	Used for	Why this one
<code>express</code>	HTTP routing	Layering stays visible to a reviewer; NestJS would add structure this size of app does not need
<code>mongoose</code>	MongoDB ODM	Schema validation, indexes and populate in one place
<code>zod</code>	Request + env validation	Parse at the edge, infer the type — one source of truth instead of a schema plus an interface
<code>bcryptjs</code>	Password hashing	Pure JS, no native build step, so <code>npm install</code> never fails on a fresh machine
<code>jsonwebtoken</code>	Access tokens	Stateless auth; refresh tokens are opaque and stored hashed
<code>luxon</code>	Timezone maths	Every schedule decision is per-user-local; <code>Date</code> cannot do this correctly
<code>node-cron</code>	Scheduled jobs	Six jobs, in-process. Redis/BullMQ would be right at multi-instance scale, not here
<code>firebase-admin</code>	Push (FCM)	Backend-driven sends to Android and iOS from one API
Email SDK	Transactional email	Verification, reset, weekly digest
<code>pino</code> + <code>pino-http</code>	Structured logging	Request IDs, and redaction so passwords never reach a log
<code>helmet</code> , <code>cors</code> , <code>express-rate-limit</code>	HTTP hardening	Standard headers, and brute-force limits on credential routes
<code>multer</code>	Avatar upload	Multipart parsing; images are stored in Mongo, not on Render's ephemeral disk

Request lifecycle

```
POST /api/v1/goals
→ helmet, cors, json body parse
→ pino-http          attaches a request id, logs start/finish
→ requireAuth        verifies the JWT, loads the user onto req.user
→ validate(schema)   zod parses body/query/params, or throws field errors
→ controller         unwraps the request, calls the service, shapes the response
→ service            the actual work: create the goal, recompute status
→ Mongoose model      writes to MongoDB
→ ok(res, data)       { success: true, message, data }
→ errorHandler        anything thrown lands here and becomes one error shape
```

Every success looks like `{ success, message, data }`. Every failure looks like `{ success: false, error: { code, message, details? } }`. The client parses exactly one shape, and `details` carries per-field messages so a form can highlight the offending input.

Environment

`src/config/env.ts` runs every variable through zod at boot. A missing or malformed value **exits the process with the key name printed**, rather than failing later on a random request. This is what surfaced the missing JWT secrets during the first Render deploy.

3. Data model

Seven collections. Everything is scoped by `user`.

```
User └─< Goal └─< Milestone └─< Action └─< ActionOccurrence
      |           └─< Action (goal-level, milestone optional)
      └─< WeeklyReflection
      └─< Notification
      └─ preferences (embedded)
      └─ notificationPreference (embedded)
      └─ refreshTokens[] (hashed, select:false)
      └─ deviceTokens[] (FCM)
```

The plan/log split

This is the decision everything else depends on.

	Role	Example
Action	the rule	"Practise Spanish, 25 min, Mon–Fri, 07:30"
ActionOccurrence	one dated instance	"Wed 13 Aug, completed at 07:52"

A recurring action has no single status — "did I miss Monday?" is unanswerable if the action is one row. Splitting them makes it a lookup:

Feature	Query
Today's actions	<code>{ user, scheduledDate: today }</code>
Missed detection	cron sets <code>upcoming</code> → <code>missed</code> once the user's day ends
Calendar	occurrences already carry a date
Weekly progress	count planned vs completed in the week
Streak	walk occurrence days backwards
Reminders	<code>scheduledAt</code> is the queue

Occurrences are never deleted, only status-changed, which is how history survives.

Indexes

Collection	Index	Purpose
<code>actionoccurrences</code>	<code>{action, scheduledDate}</code> unique	Makes materialisation idempotent — the cron can run twice without duplicating a row
<code>actionoccurrences</code>	<code>{user, scheduledDate, status}</code>	Today's feed, calendar range
<code>actionoccurrences</code>	<code>{status, scheduledAt}</code>	The 15-minute reminder sweep
<code>actionoccurrences</code>	<code>{goal, scheduledDate}</code>	Goal detail and progress
<code>goals</code>	<code>{user, status, targetDate}</code>	Goals list
<code>users</code>	<code>{email}</code> unique	Login
<code>weeklyreflections</code>	<code>{user, weekStart}</code> unique	One reflection per week

4. The scheduler

`modules/occurrences/materialiser.service.ts` turns rules into dated rows.

```

for each active goal
  for each active action
    work out which local days it falls on (daily | specific_days | weekly_count | once)
    resolve the time: action.preferredTime → goal.routine.startTime → user preference
    upsert one occurrence per day, 7 days ahead

```

Three details that matter:

- **Timezones.** Days are computed in `user.timezone` via luxon, then stored as UTC. `scheduledDate` is the UTC instant of *local* midnight, so a day-bucket query is a plain range scan and never depends on the device clock.
- **Idempotent.** `$setOnInsert` plus the unique index means re-running changes nothing.
- **Editing a routine rewrites only the future.** Upcoming occurrences from today onward are deleted and regenerated; completed and missed history is never touched.

The six cron jobs

Job	Cadence	Does
<code>materialise</code>	hourly at :10	Generate the next 7 days for every user
<code>mark-missed</code>	hourly at :20	<code>upcoming</code> → <code>missed</code> once a user's local day has ended
<code>action-reminders</code>	every 15 min	Push for occurrences due within the user's lead time
<code>daily-summary</code>	every 15 min	Fires in the bucket containing the user's chosen time
<code>weekly-digest</code>	every 30 min	Push + email on the user's chosen weekday/time
<code>recompute-goals</code>	01:00 daily	Refresh cached <code>progressPercent</code> and <code>computedStatus</code>

Jobs run **hourly, not daily**, and compare against each user's *local* clock — that is how one server serves every timezone without per-user cron entries. Each job is wrapped in a re-entrancy guard so a slow run never overlaps itself on a small instance.

5. Progress status algorithm

`modules/goals/goal.status.service.ts`. Deliberately not a percentage.

```
timeRatio = elapsed / total goal window
workRatio = completed occurrences / all planned occurrences to the target date
adherence = last 14 days: completed / (completed + missed)    [skips excluded]
delta      = workRatio - timeRatio
```

First match wins:

Condition	Status
<code>workRatio >= 1</code> or marked complete	Completed
goal younger than 7 days	On track — grace period
<code>delta >= +0.10</code>	Ahead
<code>delta >= -0.05</code> and <code>adherence >= 0.60</code>	On track
<code>delta >= -0.20</code> or <code>adherence >= 0.40</code>	Needs attention
otherwise	Behind

Notes on the choices:

- **Skips are excluded from adherence.** A skip is a decision; a miss is a failure. Counting them the same punishes honesty.
- **The denominator is projected.** Occurrences only exist 7 days ahead, so the remaining calendar is projected at the goal's own weekly cadence — otherwise every goal would look 100% complete.
- **Every status carries a sentence.** `statusReason` is generated with the status and rendered verbatim. A coloured chip alone explains nothing.

Consistency (`modules/progress/progress.service.ts`) counts a day only if something was planned for it. Days with nothing planned are *neutral* — they neither break nor extend a streak. A rest day you scheduled yourself is not a failure.

6. Notifications

The pipeline

```

cron job / login / milestone
    |
    ▼
  NotificationService.dispatch()
    |
    ├── is this type enabled for this user?
    ├── is it inside their quiet hours? (overnight ranges handled)
    |
    ├── sendPush()  FCM multicast → prunes tokens FCM rejects
    ├── sendEmail() delivers, or logs the payload in dry-run
    └── Notification document → the in-app feed and delivery audit

```

Nothing sends outside this function. Controllers and jobs never touch FCM or the email provider directly, which is why "respects user preferences" is a property of the system rather than something each caller

has to remember. Transactional mail (verification, password reset) passes `bypassPreferences` — you cannot opt out of your own reset link.

Firestore — deliberately limited

Firestore is used for **push only**, plus verifying a Google sign-in ID token. Sessions are always issued by this backend. The service account is *project*-level, so one Firestore project can serve several apps; only each mobile app needs registering separately, because `google-services.json` is tied to a package name.

Both sides degrade safely: without credentials the backend logs the payload instead of sending, and the app runs with push disabled rather than crashing at launch.

Login greeting

`modules/notifications/greeting.service.ts` — the same canned "Welcome back" every time makes an app feel like it isn't paying attention, so the message is chosen from the gap since the previous sign-in:

Gap	Kind	Message
never signed in	<code>first_time</code>	<i>Welcome to GoalFlow, Priya</i> — set your first goal
same day	<code>same_day</code>	<i>Hey again, Sam</i> — 4 things still open today
1–3 days	<code>returning</code>	<i>Welcome back, Sam</i> — 3 actions planned today, starting with Learn Spanish
4–14 days	<code>been_a_while</code>	<i>Good to see you, Sam</i> — it has been 9 days, pick one and you are moving again
15+ days	<code>long_absence</code>	<i>Welcome back, Sam</i> — 40 days away, no catching up needed, just start from today

Three implementation details:

- **Token refresh does not greet.** `issueSession` takes `{greet: false}` from `refresh()`, otherwise a notification would fire every 15 minutes.
- **First-ever login has no device token yet.** The greeting is stored and flagged `pendingGreetingAt`; registering a device flushes it (if still fresh) so the one greeting that matters most is not lost.
- **Failures are swallowed.** The whole thing is wrapped — a greeting must never be able to break signing in.

Platform setup

Platform	Needed
Android	<code>POST_NOTIFICATIONS</code> (13+), default FCM channel <code>goalflow_default</code> declared in the manifest, <code>google-services.json</code> in <code>android/app/</code>
iOS	<code>remote-notification</code> background mode, <code>Runner.entitlements</code> with <code>aps-environment</code> , <code>GoogleService-Info.plist</code> in <code>ios/Runner/</code> , APNs key uploaded to Firebase

The client handles all three delivery states: foreground (rendered via `flutter_local_notifications` , since iOS shows nothing by default), background (system tray), and terminated (`getInitialMessage` on launch). Tapping any of them reads `data.route` and navigates.

7. Authentication

Concern	Approach
Password storage	bcrypt, cost 12, <code>select: false</code> so it is never returned by accident
Access token	JWT, 15 minutes
Refresh token	48 random bytes, stored SHA-256 hashed — a database dump cannot be replayed as a session
Rotation	The presented refresh token is consumed before a new one is issued
Session limit	Newest 5 kept per account
Password change/reset	Clears every refresh token — all devices sign out
Account enumeration	Login, forgot-password and resend-code give identical answers for known and unknown emails
Brute force	Rate limiter on every credential route
Authorization	Ownership is checked per resource, not assumed from a query filter

Client side: tokens live in the platform keychain via `flutter_secure_storage` . A 401 triggers **one** transparent refresh-and-retry inside `ApiClient` ; if that fails the session is cleared and the router drops to login. No screen implements this.

8. Flutter app

Library	Used for	Why
<code>flutter_riverpod</code>	State + dependency injection	Compile-safe, testable without a widget tree, keeps logic out of widgets
<code>go_router</code>	Routing	Declarative, one auth redirect guard, deep links for free
<code>dio</code>	HTTP	Interceptors give one place for the auth header, refresh-retry and error mapping
<code>flutter_secure_storage</code>	Tokens	Keychain / Keystore, not SharedPreferences
<code>shared_preferences</code>	Theme choice	Non-sensitive device preference
<code>firebase_messaging</code> + <code>flutter_local_notifications</code>	Push	Remote delivery, and foreground rendering iOS otherwise suppresses
<code>fl_chart</code>	Progress charts	Only on the Progress screen
<code>table_calendar</code>	Schedule grid	Month view with per-day markers
<code>intl</code>	Dates and durations	Locale-aware formatting

Layers

<code>presentation/</code>	widgets and screens. No network calls, no calculations.
<code>application/</code>	Riverpod providers and notifiers. All client-side logic.
<code>data/</code>	models (hand-written fromJson) + repositories.
<code>core/</code>	theme, router, ApiClient, secure storage, push service.

No codegen. Models use hand-written `fromJson`, so a fresh clone runs with `flutter pub get` and nothing else — no `build_runner` step to forget.

How a tap flows

```
user taps the checkbox on Today
→ widget calls occurrenceActionsProvider.complete(id)
→ OccurrenceActions calls AppRepository.complete()
→ ApiClient POSTs /occurrences/:id/complete, unwraps the envelope
→ on success, invalidates dashboard, today, goals, progress, calendar, reflection
→ every affected screen refetches
```

One invalidation list is why ticking an action on Today is immediately visible on Progress and in the Calendar, with no manual cache juggling.

API URL resolution

```
--dart-define=API_BASE_URL=...    explicit, always wins
--dart-define=USE_LOCAL_API=true   localhost (10.0.2.2 on an Android emulator)
otherwise                         the deployed backend
```

Production is the **default** on purpose: an APK handed to someone else must work with no flags. Timeouts are 75 seconds because a free-tier host cold start was measured at 17 s and can reach a minute — a shorter timeout reads as a broken app.

Theming

`AppTheme.light()` / `AppTheme.dark()` are built from one function, so a colour is never defined in only one mode. Spacing, radii and type come from the `Gap` scale rather than magic numbers. The chosen mode is persisted and applied instantly — state changes first, storage is written after, so the toggle never feels laggy.

9. Testing and verification

```
cd goalflow-api && npm run typecheck    # tsc --noEmit, clean
cd goalflow_app && flutter analyze      # no issues
cd goalflow_app && flutter test         # 12 tests
```

Verified against the deployed stack: login, dashboard, complete and undo an action, progress summary, weekly reflection, calendar range, auth guard, all five greeting variants, and FCM credential authentication.

10. Known limits

Limit	Detail
Render free tier sleeps	Cron does not run while asleep. Fix: paid instance, or an external <code>/health</code> ping. The app also regenerates missing days on open, and Settings has <i>Rebuild my schedule</i>
Push needs per-app registration	<code>google-services.json</code> is bound to a package name; the backend service account is project-wide and already reusable
iOS push needs an Apple developer account	An APNs key must be uploaded to Firebase
Cron is in-process	Correct for one instance. Multiple instances would need a distributed lock or a queue
Occurrences grow over time	7 days ahead only, indexed. Old rows would eventually want archiving